MOOZO

PLAN ASSURANCE QUALITÉ

Version 1.1
31/10/2026

Présenté par
Paul DECAUCHY
Mehdi BOUYDARNE
Pierre-Alexis THIEBOT
Grégoire VAILLANT

EPITECH - T_PAR_MOOZO

Sommaire

Structure organisationnelle du projet (OBS) 3
Clarté hiérarchique 3
Cartographie des compétences 3
Définition des rôles 3

Normes 5
Git 5
Code 5

Méthodologie 7

Outils 8
Git / Github 8
Jira 8
OneDrive 8
Google drive 8
WhatsApp 8
Slack 8
Figma 8

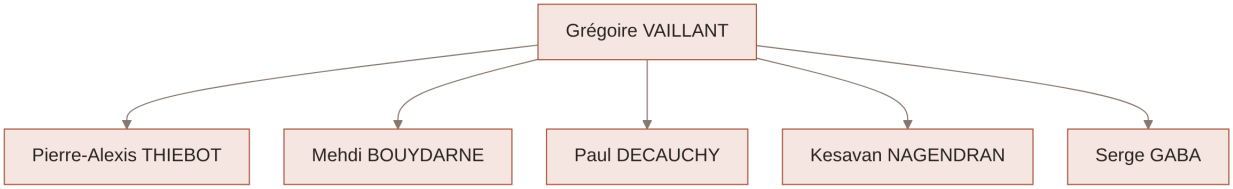
Tests 9
Tests Unitaires (TU) 9
Tests d'Intégration 9
Tests de Contrat (architecture multi-repo) 9
Tests End-to-End (E2E) 10
Critères de Qualité et Couverture 11
Récapitulatif 11
Processus d'exécution (workflow) 11

Documentation d'onboarding 12
Readme 12
Contributing 12

Choix technologiques 13
Frontend – web 13
Frontend - mobile 13
Backend (API) 13
Backend (IA) 14
Base de données 15
Infrastructure & déploiement 15
Architecture 17

Structure organisationnelle du projet (OBS)

Clarté hiérarchique



Nous adoptons une structure volontairement horizontale afin de favoriser la communication, l'autonomie et la réactivité.

Chaque membre est responsable de son périmètre technique tout en contribuant aux décisions collectives.

Cependant, afin d'éviter tout blocage décisionnel :

- Grégoire VAILLANT occupe le rôle de décisionnaire final (en cas de désaccord ou de nécessité de trancher, la décision finale est prise par Grégoire.).
- Cette organisation permet de maintenir un équilibre entre collaboration ouverte et efficacité stratégique.

Cartographie des compétences

La cartographie des compétences est maintenue dans le fichier MOOZO.xlsx, dans l'onglet dédié aux compétences de l'équipe.

Ce fichier permet d'identifier rapidement :

- Les compétences principales de chaque membre
- Les domaines d'expertise
- Les besoins d'accompagnement ou de montée en compétence
- Les personnes référentes par périmètre technique

Définition des rôles

Mehdi BOUYDARNE : Développeur opérationnel

Après 3 ans de parcours généraliste en informatique, je me suis spécialisé en Cybersécurité et Cloud.

En alternance au sein de la cellule cyber d'une PME, je travaille sur l'automatisation et l'industrialisation des déploiements, la gestion d'infrastructure et la surveillance (mise en place d'une infrastructure de collecte/monitoring, construction d'un SIEM). Je contribue également à la sécurisation de l'infrastructure (durcissement, contrôle des accès, bonnes pratiques d'exploitation).

Paul DECAUCHY : Développeur backend

Développeur backend spécialisé en Go, j'évolue actuellement chez Ubisoft au sein du Lab, une entité dédiée à l'exploration de nouveaux usages technologiques.

Je conçois des systèmes backend distribués en appliquant les principes du Domain-Driven Design (DDD), une approche qui consiste à structurer l'architecture autour du métier afin de garantir cohérence et évolutivité.

Je travaille notamment sur l'implémentation de workflows asynchrones via Uber Cadence, ce qui implique la gestion de processus complexes et de longue durée, avec des mécanismes de reprise automatique en cas d'erreur et de maintien de la cohérence des données.

J’interviens également sur l’intégration des données avec les serveurs de jeu, en veillant à assurer performance, fiabilité et qualité logicielle.

Pierre-Alexis THIEBOT : Développeur IA / ALGO

Mon profil plutôt technique. Après 3ans en développement de jeux-vidéo où j’ai développé de solides compétences en algorithmique et en optimisation, je suis actuellement en spé cybersécurité, ce apporte une compréhension des architectures, des performances et des enjeux liés à la sécurité des systèmes.

Je travaille aussi dans une startup développant un SaaS de simulation de campagnes de phishing à des fins de sensibilisation, où je touche à tout sauf au front.

J’y ai notamment développé un **evil proxy** (proxy intermédiaire capable d’intercepter et de relayer des flux pour analyser des mécanismes d’authentification dans un cadre de test), des systèmes d’envoi de campagnes de phishing simulées ainsi que des mécanismes de sécurisation. Cette expérience renforce mon intérêt pour les systèmes intelligents, l’automatisation et la compréhension fine des architectures techniques.

Même si je ne suis pas en spé IA, l’IA et la conception d’algorithmes sont des domaines qui me passionnent particulièrement.

Kesavan NAGENDRAN : Développeur Cyber sécurité et opérationnel

En alternance depuis un an et demi en tant qu’Analyste SOC, j’évolue au quotidien dans la détection, l’analyse et la réponse aux incidents de sécurité. Pour cela, je m’appuie activement sur des solutions de type SIEM, SOAR et EDR, ce qui me donne une vision très concrète des menaces actuelles et des mécanismes de défense.

Fort de cette expérience de terrain et passionné par l’audit, mon rôle au sein du projet s’apparente à celui d’un pentester. J’ai pour mission d’analyser et de décortiquer les stacks technologiques modernes que nous utilisons afin d’en comprendre précisément les rouages, les interactions et les limites.

Mon objectif principal est d’éprouver ces architectures en traquant les vulnérabilités et en identifiant les potentielles failles de sécurité. En adoptant cette approche offensive (complémentaire à mon profil défensif), je ne me contente pas de relever les problèmes : je propose également des axes d’amélioration concrets et des recommandations techniques pour durcir nos systèmes, garantissant ainsi la robustesse et la fiabilité de notre projet.

Serge GABA : Développeur Frontend Mobile

Grégoire VAILLANT : PO / Développeur full stack

Mon profil est hybride, j’ai une formation initiale en école de commerce à emlyon business school, puis une spécialisation technique en développement informatique à EPITECH. Cette double formation me permet de comprendre à la fois les enjeux stratégiques d’un projet (vision produit, priorisation, cohérence globale) et ses contraintes techniques concrètes.

Au cours de mon alternance chez Quiz Room, j’ai été impliqué à la fois dans le développement full stack (MERN, API, AWS S3, gestion serveur) et dans des échanges organisationnels et fonctionnels. Cette position intermédiaire m’a naturellement amené à faire le lien entre les besoins métier et l’implémentation technique.

Normes

Les normes de développement ont pour objectif de garantir :

- La lisibilité du code
- La maintenabilité du projet
- La cohérence entre les contributions
- La réduction des erreurs
- La simplification des revues de code
- La qualité des livrables

Ces normes portent principalement sur le contrôle de version, la structure des branches, les conventions de validation et les normes de codage pour chaque langage de programmation utilisé dans le projet.

Git

Le projet utilise Git pour la gestion de version. Le dépôt est hébergé sur GitHub afin de faciliter la collaboration, la gestion des pull requests, les revues de code et l'intégration continue.

Aucun développement ne doit être réalisé directement sur la branche principale.

Branch

Les branches utilisées sont les suivantes :

- **main** : version stable du projet
- **develop** : branche principale de développement
- **feature/nom-feature** : développement d'une nouvelle fonctionnalité
- **fix/nom-fix** : correction d'un bug
- **docs/nom-documentation** : modification documentaire
- **refactor/nom-refactor** : refactorisation sans changement fonctionnel

Toute fusion vers **develop** doit passer par une pull request validée par au moins un autre membre de l'équipe.

Commit

Les messages de commit suivent une convention inspirée de Conventional Commits :

- feat : ajout d'une fonctionnalité ;
- fix : correction d'un bug ;
- refactor : modification interne sans changement fonctionnel ;
- docs : modification de la documentation ;
- test : ajout ou modification de tests ;
- chore : tâche technique sans impact fonctionnel direct.

Exemples :

- feat : ajout de la gestion du plan de table
- fix : correction du calcul de position des invités
- docs : mise à jour du guide de contribution
- chore : Suppression de commentaires

Code

Les règles communes suivantes s'appliquent à l'ensemble du projet :

- Utiliser des noms explicites pour les variables, fonctions, composants et fichiers
- Limiter la longueur des fonctions
- Appliquer le principe de responsabilité unique
- Éviter la duplication de code
- Gérer explicitement les erreurs
- Écrire des commentaires uniquement lorsque le code n'est pas immédiatement compréhensible

- Privilégier la lisibilité à la complexité inutile
- Maintenir des fichiers courts et cohérents

Le formatage automatique est obligatoire lorsque l'écosystème du langage le permet.

GO

Les règles Go suivantes sont appliquées :

- Formatage obligatoire avec gofmt
- Noms de packages courts, en minuscules, sans underscore
- camelCase pour les variables, fonctions et méthodes non exportées
- PascalCase pour les structures, interfaces, fonctions et méthodes exportées
- Acronymes usuels conservés en majuscules dans les noms : ID, URL, HTTP, API, JSON, SQL
- Imports placés en haut du fichier (bibliothèques standard, bibliothèques externes, modules du projet)
- Séparation des imports entre bibliothèque standard, dépendances externes et modules internes
- Utilisation de structs et interfaces plutôt que de classes
- Préférence pour la composition plutôt que l'héritage
- Commentaires GoDoc obligatoires pour les éléments exportés
- Gestion systématique des erreurs
- Ajout de contexte aux erreurs avec fmt.Errorf(« …: %w », err) lorsque pertinent
- Utilisation de context.Context en premier argument pour les opérations longues, réseau ou annulables
- Un package doit avoir une responsabilité claire
- Éviter les packages génériques de type utils
- Tests placés dans des fichiers `_test.go` avec le package testing
- Gestion des dépendances avec go mod

Python

Les règles Python suivantes sont appliquées :

- snake_case pour les variables, fonctions et méthodes
- PascalCase pour les classes et exceptions
- UPPER_CASE pour les constantes
- `_leading_underscore` pour les éléments à usage interne
- Méthodes spéciales Python écrites avec la convention dunder, par exemple `__init__` ou `__str__`
- Acronymes usuels conservés en majuscules dans les noms : ID, URL, HTTP, API, SQL, DB
- Imports placés en haut du fichier
- Séparation des imports entre bibliothèque standard, dépendances externes et modules internes
- Docstrings obligatoires pour les classes et fonctions publiques
- Fonctions courtes et à responsabilité unique
- Gestion des erreurs avec des blocs try/except ciblant des exceptions précises
- Utilisation de pytest pour les tests
- Indentation 4 espaces

React

Les règles React et TypeScript suivantes sont appliquées :

- Utilisation de TypeScript pour typer les props, états, retours d'API et modèles métier
- Composants courts et réutilisables
- Séparation claire entre composants UI, logique métier, hooks, services API et types
- Organisation moléculaire des composants (Atomes, Molécules, Organismes, Templates, Pages)
- Nommage explicite des composants en PascalCase
- Nommage des hooks avec le préfixe use
- Formatage automatique avec Prettier
- Tests de composants
- Éviter la logique métier complexe directement dans les composants d'interface.

Méthodologie

Pour le développement du projet Moozo, nous avons choisi d'utiliser la méthode Scrum, issue des méthodes Agile software development. Cette approche est particulièrement adaptée aux projets logiciels nécessitant des évolutions régulières et des ajustements en fonction des retours des utilisateurs ou des nouvelles contraintes techniques.

Contrairement à la méthode Waterfall, qui repose sur une progression linéaire et séquentielle des phases (analyse, conception, développement, test), Scrum permet une organisation plus flexible basée sur des cycles courts (sprints). Cette approche facilite l'adaptation aux changements et permet de livrer régulièrement des fonctionnalités opérationnelles.

Nous prévoyons d'organiser un sprint par semaine lorsque nous avons cours une fois dans la semaine et adapterons les sprints en période d'entreprise sans cours, avec une réunion de suivi hebdomadaire permettant de faire le point sur l'avancement du projet, les tâches réalisées et les difficultés rencontrées. Cette réunion permet également de planifier les objectifs du sprint suivant.

Plusieurs livrables et outils de suivi seront produits afin de structurer l'avancement du projet :

- Compte-rendu de réunion (CR) afin de garder une trace des décisions et des points discutés
- Et Jira pour voir les statistiques liés au sprint les éléments effectués ou non en avance en retard
- Documentation technique, décrivant l'architecture et les choix technologiques du projet

Ces éléments permettent de suivre l'avancement du projet de manière structurée et de garantir une bonne coordination entre les membres de l'équipe.

Outils

Git / Github

Utilisation de Git et GitHub pour la gestion des versions du code et l'organisation des différents projets.

Jira

Jira (software) est utilisé pour la gestion des tickets, des user stories et du suivi de l'avancement des tâches.

OneDrive

Microsoft OneDrive sert au stockage et au partage des documents administratifs, cahiers des charges, rapports et fichiers bureautiques au format Word ou similaire. L'objectif est d'avoir un référentiel documentaire unique et accessible à l'équipe.

Google drive

Google Drive sert uniquement au stockage des graphiques faits avec draw.io

WhatsApp

WhatsApp est utilisé pour la communication rapide entre les membres de l'équipe pour les questions opérationnelles, la coordination ou les informations urgentes.

Slack

Slack sera utilisé pour intégrer un bot chargé de publier automatiquement des informations liées au dépôt de code ainsi qu'aux environnements de développement et de production (notifications de commits, déploiements, incidents, etc.).

Figma

Figma est utilisé pour la conception des maquettes, prototypes d'interface et validation des écrans avant la phase de développement. Les designs y sont partagés pour recueillir les retours des parties prenantes.

Tests

La stratégie de tests vise à garantir :

- La fiabilité des fonctionnalités
- La non-régression
- La cohérence entre les modules
- La validation des règles métier
- La qualité des API
- La robustesse des parcours utilisateur principaux

La stratégie s'organise en trois niveaux (pyramide de tests) : beaucoup de tests unitaires, moins de tests d'intégration, et encore moins de tests end-to-end.

Tests Unitaires (TU)

Objectif : valider la logique isolée d'un module. Les dépendances externes (API, base de données) sont simulées.

Backend — API métier (Go)

- Outil : package natif testing.
- Règle : chaque fichier des dossiers repositories et domain doit être accompagné de son fichier de test (`*_test.go`).
- Priorité : repositories MongoDB et logique métier (règles de gestion).

Microservice IA (Python)

Service indépendant du backend Go, testé de façon autonome.

- Outil : pytest.
- Périmètre : validation des algorithmes de matching de reconnaissance faciale.

Frontend (React)

- Outils : Jest + React Testing Library.
- Périmètre : logique des composants, rendu et interactions utilisateur.
- Mock des appels API : utilisation de MSW (Mock Service Worker) pour simuler les réponses du backend, plutôt que de mocker fetch à la main.

Tests d'Intégration

Objectif : vérifier la communication entre les différents modules. Le backend et la base de données sont **réels** (pas de simulation).

- Framework : github.com/ovh/venom.
- Périmètre :
  - Validation des endpoints des APIs ;
  - Vérification de la persistance des données dans MongoDB ;
  - Bon fonctionnement des middlewares.

Tests de Contrat (architecture multi-repo)

Chaque service ayant son propre repo, les tests de contrat valident les interfaces entre services sans avoir à déployer toute la stack. C'est le niveau le plus important pour fiabiliser une architecture en microservices.

- Outil : Pact (contract testing *consumer-driven*).
- Principe :
  - Le consumer (ex. Front) décrit ce qu'il attend d'une API et génère un fichier de contrat (*pact*). Le test tourne dans **son** repo.

  - Le provider (ex. Backend Go, Microservice IA) rejoue ce contrat contre son vrai code pour prouver qu'il le respecte. Le test tourne dans **son** repo.
- Élément partagé : un Pact Broker (conteneur Docker, ou PactFlow en SaaS) — pas un repo, juste un service où les contrats sont publiés puis vérifiés. Chaque CI de chaque repo dialogue avec lui.
- Avantage multi-repo : chaque test tourne isolément dans son repo, sans lancer les autres services.

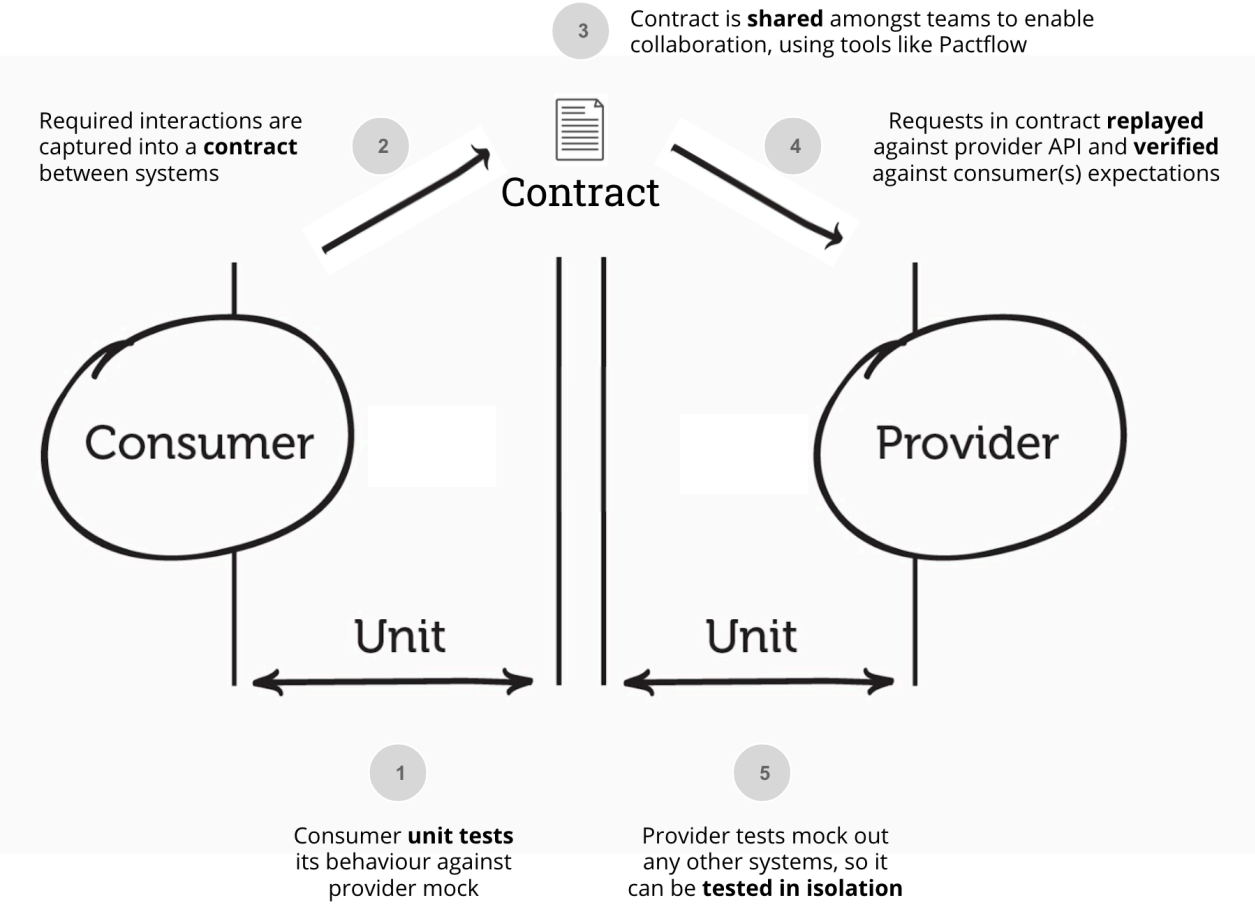


Fig. 1. – Schéma de l'organisation des tests avec Pact

Tests End-to-End (E2E)

Catégorie **transverse** : valide la chaîne complète (UI → API → MongoDB) avec tous les composants réels (**aucun mock**).

Les contrats étant déjà couverts par Pact, les E2E sont **volontairement peu nombreux** : uniquement les parcours critiques (ex. login + reconnaissance faciale).

- **Objectif :** simuler les *user stories* principales.
- **Outil :** Playwright *(à confirmer)* — navigateurs web et simulateurs mobiles.
- **Emplacement :** dans le **repo Front** (point d'entrée des parcours), pas de repo dédié.
- **Exécution :** à la main ou en CI *nightly* (lance la stack via docker-compose), distincte du pipeline unitaire.

Critères de Qualité et Couverture

Couverture de code (par périmètre)

Périmètre	Seuil minimum	Priorité / Commentaire
Backend Go	80 %	Priorité aux repositories MongoDB et à la logique métier (domain).
Microservice IA Python	65 %	Algorithmes de matching ; exclure le code de chargement de modèles.
Frontend React	65 %	Priorité à la logique des composants ; le rendu purement visuel n'est pas comptabilisé.

Critères communs (tous périmètres)

Métrique	Seuil	Commentaire
Passage des tests unitaires	100 %	Aucun merge autorisé si un test échoue.
Passage des tests d'intégration	100 %	Obligatoire pour les fonctionnalités critiques.

Récapitulatif

Niveau	Périmètre	Outil(s)	Backend
Unitaire	Backend Go	testing	mocké / isolé
Unitaire	Microservice IA Python	pytest	mocké / isolé
Unitaire	Frontend React	Jest + RTL + MSW	mocké
Intégration	Endpoints + MongoDB + middlewares	Venom	réel
Contrat	Interfaces entre services (multi-repo)	Pact + Broker	isolé par repo
End-to-End	Parcours critiques uniquement	Playwright	réel (aucun mock)

Processus d'exécution (workflow)

1. Le développeur crée une branche depuis develop.
2. Il développe la fonctionnalité ou le correctif.
3. Il exécute les tests unitaires en local.
4. Il applique le formatage obligatoire (gofmt, Prettier, etc.).
5. Il ouvre une pull request.
6. GitHub Actions déclenche automatiquement les tests.
7. Un autre membre de l'équipe effectue une revue de code.
8. La pull request est fusionnée uniquement si la CI est valide et la revue approuvée.

Documentation d’onboarding

Readme

Le fichier **Readme** présente l’objectif de l’application, son positionnement ainsi que les fonctionnalités principales du projet Moozo. Il décrit ce que l’on construit et comment le produit répond aux besoins des organisateurs, des invités et des prestataires. On y retrouve aussi les informations de démonstration, les recrutements en cours ainsi que les auteurs du projet.

Contributing

Le fichier **Contributing** explique les règles pour participer au développement du projet. Il détaille le workflow Git à respecter, les standards de code, le format des commits, le processus de pull request ainsi que les bonnes pratiques de qualité logicielle afin de garantir un développement cohérent et maintenable.

Choix technologiques

Frontend – web

React TS

Le frontend web du projet Moozo est destiné aux invités afin de leur permettre d'accéder facilement aux informations de l'événement sans installation d'application mobile. Le choix de React avec TypeScript répond à cet objectif d'accessibilité, de performance et de maintenabilité.

L'architecture component-based de React facilite le développement de ces fonctionnalités. Par exemple, le plan de table, la galerie de photos ou la liste des invités peuvent être développés sous forme de composants indépendants, réutilisables et facilement maintenables.

React utilise également un mécanisme appelé Virtual DOM, qui optimise la mise à jour de l'interface lorsque les données changent. Cela améliore la fluidité de l'expérience utilisateur dans des interfaces dynamiques où les contenus peuvent être mis à jour en temps réel (par exemple lors du partage de photos ou de l'ajout d'informations sur l'événement).

L'utilisation de TypeScript, développé par Microsoft, permet d'ajouter un typage statique au code JavaScript. Cela améliore la fiabilité de l'application et réduit les erreurs lors de la manipulation de données structurées comme les invités, les tables ou les informations d'événement. Le typage facilite également la maintenance et la collaboration sur un projet de taille importante.

Ainsi, React avec TypeScript constitue un choix adapté pour le frontend web de Moozo en offrant une interface performante, maintenable et accessible aux invités depuis n'importe quel navigateur sans installation préalable.

Frontend - mobile

Flutter

L'application mobile du projet Moozo est destinée principalement aux organisateurs et aux prestataires, qui doivent gérer l'événement directement depuis leur smartphone. Pour cela, Flutter a été choisi.

Flutter permet de développer une application mobile multiplateforme (iOS et Android) à partir d'une base de code unique, tout en conservant des performances proches du natif. Contrairement à des solutions basées sur un pont JavaScript, Flutter compile directement en code natif ARM, ce qui garantit une fluidité constante dans les interfaces dynamiques comme le plan de table ou la gestion des invités en temps réel.

Le langage Dart, typé statiquement, apporte les mêmes garanties de fiabilité que TypeScript côté web : détection des erreurs dès la compilation, meilleure maintenabilité et collaboration facilitée sur un projet manipulant de nombreuses entités (événements, invités, tables, photos, prestataires).

Flutter repose sur une architecture à base de widgets, comparable dans son principe à l'approche component-based de React utilisée côté web. Cette proximité conceptuelle (composants réutilisables, gestion d'état, cycle de vie) facilite la montée en compétence de l'équipe et limite la duplication de logique métier entre le web et le mobile, même si les deux stacks restent techniquement distinctes.

Flutter dispose également d'un écosystème mature de packages permettant d'intégrer facilement les fonctionnalités nécessaires à une application événementielle : accès à la caméra, notifications push, gestion des médias et intégration avec les services natifs des plateformes.

Ainsi, Flutter permet de développer une application mobile performante, cohérente sur iOS et Android, adaptée aux besoins des organisateurs et prestataires du projet Moozo, tout en réduisant les efforts de maintenance liés à une base de code mobile unique.

Backend (API)

Nos choix se font dans l'objectif d'avoir une api rapide et claire, un code lisible et maintenable

Golang

Performance et efficacité
Go est un langage compilé qui produit un binaire natif, ce qui lui confère des performances proches du C sans la complexité de gestion mémoire manuelle. Pour une API REST, cela se traduit par une faible latence et une consommation mémoire réduite, deux critères importants pour un service backend.

Lisibilité et maintenabilité
Go impose un style de code unique via gofmt et un ensemble de contraintes volontairement restrictives (pas de surcharge, pas d'héritage, gestion d'erreur explicite). Cela rend le code homogène et facilement lisible par tous les membres d'une équipe, ce qui est particulièrement pertinent dans un contexte de projet en groupe.

Simplicité du déploiement
Go compile en un binaire statique autonome, sans runtime externe à installer. Cela simplifie considérablement le déploiement et la conteneurisation.

Ogen

Approche OpenAPI First
Ogen adopte une approche *API First* : le fichier OpenAPI est la source de vérité unique à partir de laquelle le code serveur est entièrement généré (routing, validation, sérialisation). Cela garantit une cohérence permanente entre la documentation et l'implémentation.

Réduction du boilerplate
Le développeur n'implémente que la logique métier, sous forme d'une interface Go générée. Tout le code d'infrastructure HTTP est pris en charge par Ogen, ce qui réduit les erreurs et le temps de développement.

Sécurité par le compilateur
Si le spec OpenAPI évolue, le compilateur Go signale immédiatement les handlers manquants ou incompatibles. C'est une garantie structurelle de cohérence, plus fiable qu'une convention d'équipe.

Backend (IA)

Pourquoi l'intégrer dans un microservice séparé ?

- Le backend principal est en GO
- L'IA consomme beaucoup de CPU/RAM
- Les mises à jours du service d'IA et de l'API ne sont pas synchrone
- Rollback/scalabilité indépendant

Python
- Écosystème ML/IA dominant
- Bibliothèques optimisées
- Communauté très active

PyTorch
- Syntaxe simple et intuitive, proche du Python natif
- Très utilisé en Computer Vision
- Debug facile grâce à l'exécution en mode dynamique
- Communauté très active

Pourquoi pas TensorFlow ? Plus lourd et complexe, fait pour de gros projets avec beaucoup de stats

Base de données

MongoDB

Le projet Moozo manipule plusieurs types d'entités qui évoluent au cours du développement : utilisateurs, événements, invités, tables, photos, demandes de prestations, etc. Pour stocker ces données, nous avons retenu MongoDB.

MongoDB est une base de données orientée document : les données sont stockées sous forme de documents JSON (en BSON en interne). Ce modèle correspond bien aux objets manipulés côté API, ce qui simplifie le passage entre la base et le code applicatif sans passer par un mapping relationnel complexe.

Sur un projet en phase de conception, le modèle de données évolue régulièrement. Avec une base relationnelle, chaque modification de schéma implique une migration. MongoDB n'impose pas de schéma fixe au niveau de la base : ajouter un champ comme un statut RSVP, une préférence alimentaire ou un rôle prestataire ne demande pas de migration. La validation du schéma reste assurée côté API, ce qui garantit la cohérence des données sans bloquer les itérations.

Le modèle document est aussi adapté à des entités riches comme un événement, qui regroupe naturellement plusieurs informations liées (configuration, planning, liste d'invités, tables). Stocker ces informations dans un même document évite des jointures multiples et reste lisible à manipuler.

MongoDB dispose enfin d'un écosystème mature : drivers officiels pour Go et Python, outils d'indexation, sauvegardes et bonne intégration en environnement conteneurisé.

Pourquoi pas SQL (PostgreSQL / MySQL) ?

Une base SQL comme PostgreSQL impose un schéma strict et des migrations à chaque évolution du modèle. Cela apporte une garantie d'intégrité forte, mais ralentit les itérations sur un projet où la structure des entités n'est pas encore figée. Pour Moozo, la flexibilité de MongoDB et la rapidité de développement priment sur la rigidité d'un schéma relationnel.

Infrastructure & déploiement

Docker

Docker est utilisé pour conteneuriser les services applicatifs de Moozo, notamment l'API Go et le microservice IA Python. Chaque service peut ainsi être empaqueté avec ses dépendances, ce qui garantit un comportement plus stable entre les environnements de développement, de test et de déploiement.

En local, Docker permet de lancer facilement la stack de développement avec docker-compose, incluant l'API, le service IA et une instance MongoDB de test. En environnement cloud, seuls les services applicatifs sont déployés sous forme de conteneurs, tandis que la base de données est hébergée séparément via MongoDB Atlas.

Ce choix simplifie le travail en équipe, limite les problèmes de configuration locale et s'intègre naturellement avec la CI/CD. Les images Docker peuvent être construites automatiquement avec GitHub Actions, stockées dans un registre, puis déployées sur Azure Container Apps.

Azure

Azure est retenu comme socle cloud principal pour l'infrastructure Moozo. L'objectif n'est pas de mettre en place une infrastructure inutilement complexe, mais de déployer le projet sur un environnement cloud moderne, managé et proche de ce qui est utilisé en entreprise.

Ce choix permet de travailler des sujets importants pour le projet : hébergement applicatif, conteneurisation, stockage des médias, gestion des secrets, supervision, logs et automatisation des déploiements. Azure permet donc de couvrir une grande partie des besoins d'exploitation de Moozo sans devoir administrer manuellement des machines virtuelles.

Ce choix est aussi motivé par la montée en compétence. Moozo est un projet de fin d'études, et l'infrastructure peut servir de support concret pour progresser sur des compétences recherchées : cloud, DevOps, CI/CD, sécurité des accès, monitoring et gestion de services managés. Utiliser Azure permet donc de faire du projet un vrai cas pratique, au-delà d'un simple déploiement local ou d'un VPS classique.

Enfin, Azure reste un choix cohérent pour une application destinée à des utilisateurs français, car il permet d'héberger les ressources dans des régions proches, notamment en France ou en Europe. Cela permet de limiter la latence et de mieux maîtriser la localisation des données, un sujet de plus en plus pris en compte dans les projets numériques.

Azure Static Web Apps

Azure Static Web Apps est utilisé pour héberger le frontend web de Moozo. Cette interface permet aux utilisateurs d'accéder aux fonctionnalités de la plateforme depuis un navigateur, selon leur rôle : invité, organisateur, prestataire ou administrateur. Le service est adapté à une application React TS, avec un déploiement simple depuis GitHub, la gestion du HTTPS et la possibilité d'associer un nom de domaine personnalisé.

Azure Container Apps

Azure Container Apps est utilisé pour exécuter l'API Go et le microservice IA Python. Ce service permet de déployer des conteneurs sans gérer directement une machine virtuelle ou un cluster Kubernetes. C'est adapté à Moozo, car l'API et le service IA peuvent rester séparés, tout en étant déployés et redimensionnés indépendamment.

En MVP, le mode Consumption permet de limiter les coûts avec une facturation à l'usage. Si l'application devient plus utilisée, le mode Dedicated pourra être envisagé pour une charge plus stable.

Azure Blob Storage

Azure Blob Storage est utilisé pour stocker les photos, les plans de salle et les médias liés aux événements. Ces fichiers ne doivent pas être stockés directement dans MongoDB ni sur le disque local d'un serveur. Le stockage objet est plus adapté pour ce type de données, car il est conçu pour gérer des fichiers volumineux et évolutifs.

Azure Key Vault

Azure Key Vault est utilisé pour stocker les secrets applicatifs : clés Stripe, URI MongoDB, tokens JWT, clés API, identifiants email ou SMS. Cela évite de mettre des secrets dans le code, dans des fichiers .env partagés ou dans les variables de pipeline sans contrôle. C'est une brique simple mais importante pour rendre l'infrastructure plus propre et plus sécurisée.

Azure Monitor + Log Analytics

Azure Monitor et Log Analytics sont utilisés pour centraliser les logs, suivre les erreurs et créer des alertes. L'objectif est d'avoir une application exploitable : savoir si l'API renvoie des erreurs, si le microservice IA plante, ou si un comportement anormal apparaît. Ce n'est pas un SIEM complet, mais c'est suffisant pour une première vraie supervision cloud.

GitHub Container Registry / Azure Container Registry

GitHub Container Registry est utilisé dans un premier temps pour stocker les images Docker de l'API Go et du microservice IA Python. Ce choix est cohérent avec la phase de développement du projet, car le code est déjà versionné sur GitHub et les images peuvent être construites automatiquement avec GitHub Actions.

Dans le cadre du projet Moozo, même si l'application est déployée sur Azure, GitHub Container Registry reste suffisant pour le MVP. Il permet de construire, versionner et stocker les images Docker sans ajouter une brique supplémentaire trop tôt dans l'architecture.

Azure Container Registry devient surtout pertinent si Moozo évolue vers une application réellement exploitée ou commercialisée. Dans ce cas, il serait plus logique de stocker les images Docker directement dans l'écosystème Azure, afin de mieux les intégrer avec Azure Container Apps, la gestion des accès, la supervision et les règles de sécurité de l'environnement cloud.

Le choix retenu est donc progressif : GitHub Container Registry pour le développement, le MVP et le déploiement du projet, puis Azure Container Registry en évolution possible si l'application devient un produit réellement exploité.

Azure CDN / Front Door

Azure CDN ou Front Door n'est pas nécessaire dès le MVP. Ce service peut être ajouté plus tard si les galeries photos deviennent fortement utilisées. Son rôle serait d'accélérer le chargement des fichiers statiques et des médias, notamment les photos partagées pendant les événements.

Azure Communication Services Email & SMS

Azure Communication Services est retenu pour gérer les communications transactionnelles de Moozo. La partie **Email** couvre les emails de vérification de compte, de réinitialisation de mot de passe, d'invitation RSVP, de relance et de notification. La partie **SMS** couvre les messages liés aux RSVP, aux relances et éventuellement à certains codes de vérification.

Ce choix permet de rester dans l'écosystème Azure au lieu d'ajouter des services externes comme Brevo pour l'email ou OVHcloud SMS pour les SMS. Il est donc plus intéressant dans le cadre du projet, car il permet de travailler une brique supplémentaire d'Azure : création du service, configuration du domaine d'envoi, intégration API, gestion des clés, suivi des envois et supervision.

Pour Moozo, l'email et le SMS ne sont pas le cœur du produit, mais ils restent nécessaires dans plusieurs parcours utilisateurs. Utiliser Azure Communication Services permet de couvrir ces besoins sans mettre en place de serveur SMTP auto-hébergé ni multiplier les prestataires externes.

Le SMS reste cependant un coût variable important, car il dépend directement du nombre de messages envoyés. C'est donc un point à prendre en compte dans l'estimation des coûts, surtout dans les scénarios de croissance et de forte utilisation.

MongoDB Atlas

MongoDB Atlas est retenu pour héberger la base de données Moozo. Ce choix reste cohérent avec l'utilisation de MongoDB déjà prévue dans le projet pour stocker les utilisateurs, événements, invités, RSVP, prestations, messages et données liées à l'organisation des événements.

Atlas évite d'avoir à administrer nous-mêmes un serveur MongoDB. Le service permet de commencer avec une offre gratuite, puis de passer sur une offre plus robuste si le projet prend de l'ampleur.

GitHub Actions

GitHub Actions est utilisé pour automatiser la CI/CD du projet. Il permet de lancer les tests, vérifier le formatage, construire les images Docker et déclencher les déploiements vers les environnements prévus.

Ce choix est cohérent avec l'utilisation de GitHub pour le versionnement du code. Il permet de garder un workflow simple : les développeurs ouvrent des pull requests, la CI vérifie automatiquement le projet, puis le déploiement peut être lancé de manière contrôlée. Des outils de sécurité peuvent aussi être ajoutés dans le pipeline, comme des scans de secrets, d'images Docker ou de fichiers d'infrastructure.

Registrar externe

Un registrar externe est retenu pour l'achat du nom de domaine et la gestion DNS. Azure n'apporte pas de valeur particulière pour cette partie du projet. Utiliser un fournisseur spécialisé reste simple, peu coûteux et facilement interchangeable.

Le domaine pourra ensuite pointer vers les services hébergés sur Azure, notamment le frontend web. Cela permet de séparer proprement l'hébergement applicatif de la gestion du nom de domaine.

Architecture

L'infrastructure de Moozo repose entièrement sur Azure. Les services applicatifs — l'API Go et le microservice de reconnaissance faciale — sont conteneurisés et exécutés sur Azure Container Apps. Le point d'entrée public est l'Application Gateway, équipée d'un pare-feu applicatif (WAF), qui assure la terminaison TLS et le routage vers l'API. Les composants applicatifs, la base de données, le stockage et les secrets sont déployés dans un réseau virtuel privé (VNet) et accessibles uniquement par liaisons privées (Private Link). L'infrastructure est décrite et déployée en Infrastructure as Code avec Terraform.

Caractéristiques principales :

- **Exposition** : un seul point d'entrée public (Application Gateway + WAF). Aucun autre service n'expose d'adresse publique.
- **Communication interne** : authentification entre services par identité managée (Managed Identity) et contrôle d'accès RBAC. Les secrets sont centralisés dans Azure Key Vault.
- **Microservice IA** : déployé en interne au VNet et appelé uniquement par l'API. Les photos sont chiffrées au repos et hébergées en région France.
- **Paiements** : délégués à Stripe.

Vue d'ensemble

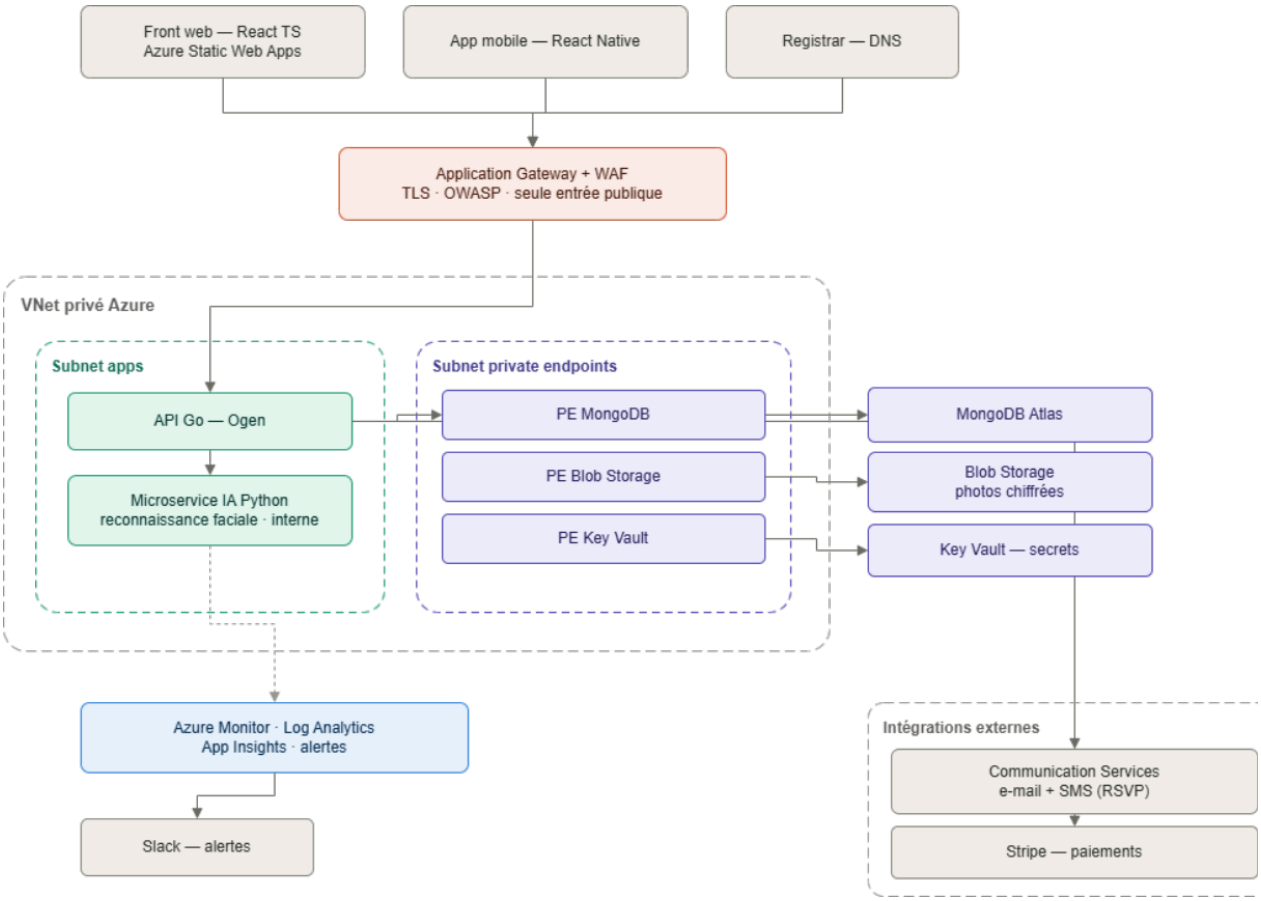


Fig. 2. – Architecture technique

Lien vers le draw.io

Segmentation réseau (VNet)

Région : France Central. VNet 10.20.0.0/16 segmenté par sous-réseaux, chacun doté de son groupe de sécurité réseau (NSG).

Sous-réseau	CIDR	Rôle	Contenu
snet-appgw	10.20.0.0/24	Périmètre public	Application Gateway + WAF (IP publique)
snet-apps	10.20.1.0/24	Compute applicatif	Environnement Azure Container Apps (API Go, IA Python)
snet-pe	10.20.2.0/24	Accès données privé	Private endpoints (MongoDB, Blob, Key Vault)

Des zones DNS privées associées au VNet font résoudre les noms managés vers les adresses privées des endpoints. Les environnements staging et prod réutilisent les mêmes modules avec un plan d'adressage distinct.

Exposition & flux

Élément	Exposition	Détail
Application Gateway + WAF	Public	Unique IP publique, termine le TLS, filtre OWASP, route vers l'API.
Front web (Static Web Apps)	Public (CDN managé)	Sert les fichiers statiques React ; le navigateur appelle ensuite l'API.
API Go	Privé	Ingress interne au VNet, atteignable uniquement derrière l'App Gateway.
Microservice IA	Privé strict	Appelé uniquement par l'API Go ; jamais exposé.
MongoDB / Blob / Key Vault	Privé (Private Link)	Aucun accès Internet ; trafic via private endpoints.
Stripe, Communication Services	Egress sortant	Appels sortants contrôlés depuis l'API.

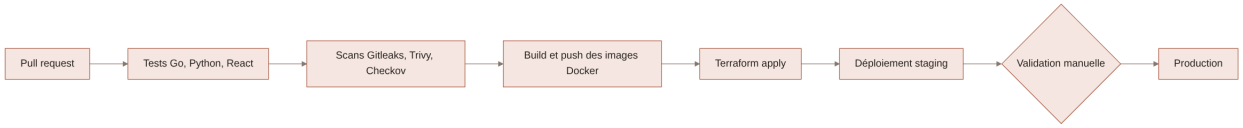
Observabilité

L'API et le microservice IA sont instrumentés via Application Insights (logs, métriques et traces), centralisés dans un espace de travail Log Analytics. Les alertes Azure Monitor sont routées vers Slack et réparties par catégorie : sécurité, ressources et disponibilité. Les sauvegardes reposent sur les snapshots planifiés de MongoDB Atlas et le soft-delete avec versioning du Blob Storage.

Sécurité & moindre privilège (points de contrôle)

Flux	Contrôle
Internet → API	Application Gateway + WAF (OWASP), TLS, seule entrée publique.
API → Key Vault / Blob / MongoDB	Private Link + Managed Identity : pas de secret en clair, pas de trafic Internet.
API → microservice IA	Interne au VNet uniquement, jamais exposé ; appel restreint à l'API.
Photos / données biométriques	Chiffrement au repos (Blob), région France, durée de conservation limitée.
Paiements	Délégués à Stripe.
Secrets	Azure Key Vault, injectés au runtime via Managed Identity ; rien en clair dans git.
Firewall réseau	NSG deny par défaut par sous-réseau ; seuls les flux nécessaires sont autorisés.

CI/CD & Infrastructure as Code



Une pull request sur develop ou main déclenche GitHub Actions : tests (Go, Python, React) et scans de sécurité (Gitleaks, Trivy, Checkov). Si tout passe, les images Docker sont construites et poussées dans le registre, puis Terraform provisionne l'infrastructure Azure et le déploiement va sur staging, avec validation manuelle avant la production. Aucun merge n'est autorisé si la CI échoue.

Chaque bloc du schéma de la section 1 correspond à un module Terraform :

```
terraform/
├─ environments/
│ ├─ staging/ (main.tf, terraform.tfvars, backend.tf)
│ └─ prod/ (main.tf, terraform.tfvars, backend.tf)
├─ modules/
│ ├─ network/ VNet, subnets, NSG, zones DNS privées
│ ├─ app_gateway/ Application Gateway + WAF + IP publique + TLS
│ ├─ container_apps/ environnement interne + API Go + IA Python
│ ├─ storage/ compte de stockage + Blob + private endpoint
│ ├─ keyvault/ Key Vault + private endpoint + RBAC
│ ├─ identity/ managed identities + role assignments
│ ├─ observability/ Log Analytics + App Insights + alertes
│ └─ communication/ Azure Communication Services (e-mail / SMS)
├─ providers.tf provider azurerm
└─ versions.tf versions Terraform & providers épinglées
```

Le state Terraform est stocké à distance dans un Blob Azure dédié (verrouillage natif), jamais en local dans git. Ajouter un environnement revient à créer un dossier dans environments/ réutilisant les mêmes modules avec son propre plan d'adressage.

Reproductibilité

- terraform init / plan / apply par environnement ; state distant dans un Blob Azure.
- Détruire puis reconstruire : terraform destroy puis terraform apply recrée toute l'infra ; le pipeline GitHub Actions redéploie ensuite les images applicatives.
- Un nouvel environnement = un dossier dans environments/ réutilisant les mêmes modules.